AgileFlow AI

Human-in-the-Loop Multi-Agent Framework for Agile SDLC Automation

File 06 - API and Data Schema Plan

Purpose	Define the common backend API contracts, shared data models, MongoDB collections, workflow statuses, approval gates, traceability IDs, and integration rules required to start AgileFlow AI implementation without duplicated functions or broken integration.

1. API and Schema Design Goal

This document acts as the technical contract for the AgileFlow AI implementation team. It should be used before coding any backend endpoint, frontend API call, database document, or AI-generated module. The goal is to keep all four components aligned through one shared API and data structure plan.

- Use one common project ID across all agents and artifacts.
- Use fixed approval gates H1 to H5 for human-in-the-loop control.
- Store every agent output in the Shared Knowledge Base through consistent collections.
- Preserve traceability from requirement to design, test, code, bug, security finding, and final report.
- Avoid duplicate models, duplicate endpoints, inconsistent status names, and missing integration functions.

2. Architecture Assumption for APIs

The recommended implementation is a React frontend with a FastAPI backend and MongoDB. The backend exposes REST APIs first. LLM, Jira, GitHub, Docker, and security scanners can be integrated later after the workflow is stable.

Layer	Recommended Technology	Role in System
Frontend	React + Vite + Tailwind CSS	Agent dashboards, approval screens, workflow visualization, reports
Backend	Python FastAPI	REST APIs, agent service coordination, orchestration, data validation
Database	MongoDB	Shared Knowledge Base for JSON artifacts and workflow state
File Storage	Local /storage first	Uploaded files, generated artifacts, logs, reports
Future Integrations	Jira, GitHub, Docker, testing tools, security scanners	Start as mock services, then replace with real integrations

3. Common Naming and ID Standards

All agents must follow the same ID patterns. This is essential for traceability and final report generation.

Artifact Type	ID Format	Example
Project	PRJ-###	PRJ-001
Requirement	REQ-###	REQ-001
User Story	US-###	US-001
Acceptance Criteria	AC-###	AC-001
Design Artifact	DES-###	DES-001
API Endpoint Spec	API-###	API-001

Database Entity	DB-###	DB-001
Backlog Item / Task	TASK-###	TASK-001
Test Case	TEST-###	TEST-001
Code Module	CODE-###	CODE-001
Bug Report	BUG-###	BUG-001
Security Finding	SEC-###	SEC-001
Traceability Link	TRACE-###	TRACE-001

4. Common Status Values

Do not allow each member or AI coding agent to invent separate status values. These values should be used in frontend badges, backend models, workflow events, approval gates, and reports.

```
PENDING
IN_PROGRESS
WAITING_FOR_APPROVAL
APPROVED
REJECTED
REQUEST_CHANGES
FAILED
RETRYING
COMPLETED
```

5. Human Approval Gates

Gate	Approval Point	Main Owner	Moves Workflow To
H1	Requirement approval	Requirement Gathering and Analysis Agent	Project Management and Design Agent
H2	Planning and design approval	Project Management and Design Agent	Pre-Code Test Planning Agent
H3	Pre-code test package approval	Pre-Code Test Planning Agent	Implementation and Git Automation Agent
H4	Implementation/prototype approval	Implementation and Git Automation Agent	Testing, Validation and Security Agent
H5	Final validation approval	Testing, Validation and Security Agent	Final Reports

6. Main API Route Groups

All endpoints use /api/v1 as the base path. Project ID is passed as a path parameter when the endpoint is related to one project workflow.

```
Base URL: /api/v1
Project-scoped pattern: /api/v1/projects/{project_id}/...
Response format: JSON
Request format: JSON or multipart/form-data for uploads
Authentication: optional in MVP, add later if needed
```

7. Project and Workflow APIs

Method	Endpoint	Purpose	Owner
POST	/api/v1/projects	Create a new AgileFlow AI project	Core/Foundation
GET	/api/v1/projects	List projects for dashboard	Core/Foundation
GET	/api/v1/projects/{project_id}	Get project details and current state	Core/Foundation
PATCH	/api/v1/projects/{project_id}	Update basic project metadata	Core/Foundation

GET	/api/v1/projects/{project_id}/workflow	Get full workflow state and gate status	Orchestrator
POST	/api/v1/projects/{project_id}/orchestrator/next-step	Move workflow to next allowed stage	Orchestrator
POST	/api/v1/projects/{project_id}/orchestrator/route-failure	Route test, build, security, or requirement failure	Orchestrator
GET	/api/v1/projects/{project_id}/events	Get workflow activity timeline	Orchestrator

8. Requirement Gathering and Analysis APIs

Method	Endpoint	Purpose
POST	/api/v1/projects/{project_id}/inputs	Upload or submit client input sources
GET	/api/v1/projects/{project_id}/inputs	List uploaded/submitted input sources
POST	/api/v1/projects/{project_id}/requirements/process	Generate structured requirements from client inputs
GET	/api/v1/projects/{project_id}/requirements	Get generated requirements, user stories and acceptance criteria
POST	/api/v1/projects/{project_id}/requirements/clarifications	Add clarification questions or human feedback
POST	/api/v1/projects/{project_id}/approvals/h1	Approve, reject, or request changes for requirements

9. Project Management and Design APIs

Method	Endpoint	Purpose
POST	/api/v1/projects/{project_id}/design/generate	Generate planning and design package from approved requirements
GET	/api/v1/projects/{project_id}/backlog	Get epics, user stories, tasks, subtasks, bugs
GET	/api/v1/projects/{project_id}/design/artifacts	Get UML, API spec, DB schema, UI flows, wireframe plan
GET	/api/v1/projects/{project_id}/design/traceability	Get requirement-to-design mapping
POST	/api/v1/projects/{project_id}/design/jira-sync	Mock or future real Jira sync
POST	/api/v1/projects/{project_id}/approvals/h2	Approve, reject, or request changes for planning/design

10. Testing, Validation and Security APIs

Agent 4 appears in two workflow phases: pre-code test planning before implementation and post-code validation after implementation.

Phase	Method	Endpoint	Purpose
Pre-Code	POST	/api/v1/projects/{project_id}/tests/pre-code/generate	Generate test specifications and test skeletons
Pre-Code	GET	/api/v1/projects/{project_id}/tests/pre-code	Get approved or draft pre-code test package
Pre-Code	POST	/api/v1/projects/{project_id}/approvals/h3	Approve, reject, or request changes for test package
Post-Code	POST	/api/v1/projects/{project_id}/validation/run-tests	Run or simulate test execution
Post-Code	POST	/api/v1/projects/{project_id}/validation/security-scan	Run or simulate security validation
Post-Code	GET	/api/v1/projects/{project_id}/validation/report	Get final test and security validation report
Post-Code	POST	/api/v1/projects/{project_id}/approvals/h5	Approve, reject, or request final validation changes

11. Implementation and Git Automation APIs

Method	Endpoint	Purpose
POST	/api/v1/projects/{project_id}/implementation/generate	Generate or simulate backend/frontend prototype modules
GET	/api/v1/projects/{project_id}/implementation/modules	Get generated code module metadata
POST	/api/v1/projects/{project_id}/implementation/git/init	Mock or future real GitHub repository setup
POST	/api/v1/projects/{project_id}/implementation/run	Run or simulate Docker/build execution
GET	/api/v1/projects/{project_id}/implementation/logs	Get build/runtime logs
POST	/api/v1/projects/{project_id}/implementation/self-fix	Apply limited self-fix attempt for build/runtime issue
POST	/api/v1/projects/{project_id}/approvals/h4	Approve, reject, or request changes for generated implementation

12. Final Reports and Traceability APIs

Method	Endpoint	Purpose
GET	/api/v1/projects/{project_id}/traceability	Get requirement-to-design-to-test-to-code-to-validation matrix
POST	/api/v1/projects/{project_id}/traceability/rebuild	Rebuild traceability links from current artifacts
GET	/api/v1/projects/{project_id}/reports/final	Get final report summary
GET	/api/v1/projects/{project_id}/reports/export	Export reports as PDF/DOCX/JSON later
GET	/api/v1/projects/{project_id}/artifacts	List all stored artifacts in Shared Knowledge Base

13. Shared Data Schemas

The following schemas are the first common contract for implementation. Exact Pydantic models can be created from these structures.

Project

```
{
  "project_id": "PRJ-001",
  "name": "Book Management System",
  "domain": "Library / Education",
  "description": "Client idea or project summary",
  "current_stage": "REQUIREMENT_GATHERING",
  "status": "IN_PROGRESS",
  "created_by": "user_id",
  "created_at": "ISO_DATE",
  "updated_at": "ISO_DATE"
}
```

Requirement

```
{
  "requirement_id": "REQ-001",
  "project_id": "PRJ-001",
  "title": "Manage books",
  "description": "System shall allow authorized users to add, edit and delete books.",
  "type": "FUNCTIONAL",
  "priority": "HIGH",
  "source_reference": "input_id or transcript segment",
  "quality_flags": ["AMBIGUOUS", "MISSING_DETAIL"],
  "status": "WAITING_FOR_APPROVAL"
}
```

UserStory

```
{
  "story_id": "US-001",
  "project_id": "PRJ-001",
  "requirement_id": "REQ-001",
  "role": "Librarian",
  "goal": "manage book records",
  "benefit": "keep the catalog updated",
  "priority": "HIGH"
}
```

AcceptanceCriteria

```
{
  "criteria_id": "AC-001",
  "project_id": "PRJ-001",
  "requirement_id": "REQ-001",
  "story_id": "US-001",
  "given": "A librarian is logged in",
  "when": "the librarian adds valid book details",
  "then": "the system saves the book and shows it in the catalog"
}
```

DesignArtifact

```
{
  "design_id": "DES-001",
  "project_id": "PRJ-001",
  "artifact_type": "API_SPEC | DB_SCHEMA | UML | WIREFRAME | UI_FLOW",
  "title": "Book API specification",
  "linked_requirement_ids": ["REQ-001"],
  "content": {},
  "status": "WAITING_FOR_APPROVAL"
}
```

BacklogItem

```
{
  "task_id": "TASK-001",
  "project_id": "PRJ-001",
  "item_type": "EPIC | STORY | TASK | SUBTASK | BUG",
  "title": "Create book CRUD API",
  "description": "Implement APIs for book management",
  "linked_requirement_ids": ["REQ-001"],
  "priority": "HIGH",
  "sprint": "Sprint 1",
  "status": "PENDING"
}
```

TestCase

```
{
  "test_id": "TEST-001",
  "project_id": "PRJ-001",
  "test_type": "UNIT | API | INTEGRATION | UI | SECURITY | REGRESSION",
  "title": "Add book with valid data",
  "linked_requirement_ids": ["REQ-001"],
  "preconditions": ["User is authenticated"],
  "steps": ["Open add book form", "Enter valid details", "Submit"],
  "expected_result": "Book is saved successfully",
  "execution_status": "PENDING"
}
```

CodeModule

```
{
  "code_id": "CODE-001",
  "project_id": "PRJ-001",
  "module_name": "books_api",
  "module_type": "BACKEND | FRONTEND | CONFIG | DATABASE",
  "file_paths": ["backend/app/modules/books/router.py"],
  "linked_requirement_ids": ["REQ-001"],
  "branch_name": "feature/books-api",
  "commit_hash": "mock-or-real-hash",
  "build_status": "PASSED"
}
```

BugReport

```
{
  "bug_id": "BUG-001",
  "project_id": "PRJ-001",
  "title": "Add book API returns 500",
  "severity": "HIGH",
  "source": "TEST_EXECUTION",
  "linked_test_id": "TEST-001",
  "linked_requirement_ids": ["REQ-001"],
  "assigned_to_agent": "Implementation and Git Automation Agent",
  "status": "OPEN"
}
```

SecurityFinding

```
{
  "security_id": "SEC-001",
  "project_id": "PRJ-001",
  "title": "Missing authorization check",
  "severity": "CRITICAL",
  "category": "AUTHORIZATION",
  "linked_code_id": "CODE-001",
  "recommendation": "Add role-based authorization before modifying books.",
  "status": "OPEN"
}
```

Approval

```
{
  "approval_id": "APP-001",
  "project_id": "PRJ-001",
  "gate": "H1",
  "artifact_type": "REQUIREMENTS",
  "decision": "APPROVED | REJECTED | REQUEST_CHANGES",
  "reviewer_comment": "Approved after clarification.",
  "reviewed_by": "human_user_id",
  "reviewed_at": "ISO_DATE"
}
```

TraceabilityLink

```
{
  "trace_id": "TRACE-001",
  "project_id": "PRJ-001",
  "requirement_id": "REQ-001",
  "design_ids": ["DES-001"],
  "task_ids": ["TASK-001"],
  "test_ids": ["TEST-001"],
  "code_ids": ["CODE-001"],
  "bug_ids": ["BUG-001"],
  "security_ids": ["SEC-001"],
  "current_status": "VALIDATED"
}
```

14. MongoDB Collections

Collection	Stores	Used By
projects	Project metadata and current workflow stage	All modules
client_inputs	Text, document, link, audio/video metadata	Requirement Agent
requirements	Requirements, user stories, acceptance criteria, quality issues	Agent 1, Agent 2, Agent 4, Agent 3
design_artifacts	UML, API spec, DB schema, UI flows, wireframe plans	Agent 2, Agent 3, Agent 4
backlog_items	Epics, stories, tasks, subtasks, bugs	Agent 2, Orchestrator
test_cases	Pre-code and executable test cases	Agent 4, Agent 3
implementation_artifacts	Code modules, generated files, Git metadata	Agent 3, Agent 4
execution_logs	Build logs, Docker logs, runtime logs, retry logs	Agent 3, Agent 4, Orchestrator
test_results	Passed/failed/skipped tests and coverage summary	Agent 4
security_findings	Vulnerability findings and recommendations	Agent 4, Agent 3, Agent 2
bug_reports	Failed tests and defect records	Agent 4, Agent 2, Agent 3
approvals	Human approval gate decisions H1-H5	Orchestrator, all agents
traceability_links	End-to-end artifact mappings	Reports, all agents
workflow_events	Timeline of actions and transitions	Orchestrator

15. Orchestrator Workflow Stage Names

```
PROJECT_CREATED
REQUIREMENT_GATHERING
WAITING_H1_APPROVAL
DESIGN_PLANNING
WAITING_H2_APPROVAL
PRE_CODE_TEST_PLANNING
WAITING_H3_APPROVAL
IMPLEMENTATION_GENERATION
WAITING_H4_APPROVAL
POST_CODE_VALIDATION
WAITING_H5_APPROVAL
FINAL_REPORT_READY
COMPLETED
```

16. Standard Response Format

All APIs should follow one response style to make frontend integration easier.

```
{
  "success": true,
  "message": "Operation completed successfully",
  "data": {},
  "errors": [],
  "workflow_stage": "DESIGN_PLANNING",
  "next_action": "WAITING_H2_APPROVAL"
}
```

For errors:

```
{
  "success": false,
  "message": "Validation failed",
  "data": null,
  "errors": [
```

```
    { "field": "project_id", "message": "Project not found" }
  ],
  "workflow_stage": "REQUIREMENT_GATHERING",
  "next_action": "FIX_INPUT"
}
```

17. Frontend API Service File Plan

The frontend should not call fetch/axios randomly inside every page. Create one service file per module.

Frontend Service File	Purpose
frontend/src/services/projectApi.js	Project create/list/details APIs
frontend/src/services/workflowApi.js	Workflow state, next step, events
frontend/src/services/requirementsApi.js	Agent 1 requirement APIs
frontend/src/services/designApi.js	Agent 2 design and backlog APIs
frontend/src/services/testingApi.js	Agent 4 pre-code and post-code APIs
frontend/src/services/implementationApi.js	Agent 3 implementation and log APIs
frontend/src/services/reportApi.js	Traceability and final reports APIs

18. Backend Folder Structure Linked to APIs

```
backend/app/
├── main.py
├── core/
│   ├── config.py
│   └── constants.py
├── database/
│   └── mongodb.py
├── schemas/
│   ├── common.py
│   ├── project.py
│   ├── requirement.py
│   ├── design.py
│   ├── testing.py
│   ├── implementation.py
│   ├── approval.py
│   └── traceability.py
├── modules/
│   ├── projects/
│   ├── orchestrator/
│   ├── requirements/
│   ├── design/
│   ├── testing/
│   ├── implementation/
│   └── reports/
└── utils/
```

19. Member Ownership for APIs and Schemas

Member / Component	Owns APIs	Owns Schemas
Group Leader / Foundation	Projects, workflow, approvals, orchestrator, reports base	Project, WorkflowEvent, Approval, TraceabilityLink
Requirement Gathering and Analysis Agent	Client inputs, requirements, clarifications, H1	Requirement, UserStory, AcceptanceCriteria, RequirementQualityIssue
Project Management and Design Agent	Design generation, backlog, Jira mock, H2	DesignArtifact, BacklogItem, ApiSpec, DbSchema
Implementation and Git Automation Agent	Implementation generation, Git mock, run logs, self-fix, H4	CodeModule, GitMetadata, ExecutionLog
Testing, Validation and Security Agent	Pre-code tests, test run, security scan, H3, H5	TestCase, TestResult, BugReport, SecurityFinding, ValidationReport

20. Rules for AI Coding Agents

- AI agents must use the endpoint names and schemas defined in this document.
- AI agents must not create duplicate models if a shared model already exists.
- AI agents must not rename fields such as project_id, requirement_id, status, gate, or traceability links.
- AI agents must not directly edit another component folder unless the group leader approves.
- Any endpoint change must update API_CONTRACT.md and related frontend service file.
- Any schema change must update DATA_SCHEMAS.md and any affected seed/sample data.
- The develop branch must remain runnable after every merge.

21. First Implementation Priority

1. Create project APIs and MongoDB connection.
2. Create workflow state and approval gate APIs.
3. Create frontend service files using the API plan.
4. Build Agent 1, Agent 2, Agent 4 pre-code, Agent 3, Agent 4 post-code APIs in sequence.
5. Connect every output to traceability links before final report generation.
6. Use mock data first; replace with real AI/Git/Docker/scanner logic later.

22. Final Checklist Before Members Start Coding

Checklist Item	Status
API endpoints reviewed and accepted by all members	Required
Common schema names finalized	Required
Status values finalized	Required
Approval gate names finalized	Required
MongoDB collection names finalized	Required
Frontend service files created	Required
Branch ownership assigned	Required
AI agent rules shared with all members	Required